Ticket 04 Study Report: PSOR/LCP Core for American Option Obstacle

SURF2026 American Option Risk Surfaces

Codex-assisted implementation report

June 16, 2026

Abstract

Ticket 04 adds the first American-option numerical core: projected successive over-relaxation, or PSOR, for the linear complementarity problem created by the American payoff obstacle. It also adds a basic Crank-Nicolson/PSOR time-marching solver for American puts and dividend-paying American calls. This is a reusable solver layer, not the full American put validation study. The report explains the finance and numerical ideas slowly for beginner researchers who know Python and data work, but are still learning variational inequalities, LCPs, PSOR, and computational finance.

Contents

1 Role of Ticket 04 in the Whole Solver Project

Tickets 01 through 03 built the analytic and European finite-difference foundation. Ticket 01 implemented payoff and European Black-Scholes closed-form utilities. Ticket 02 implemented the uniform grids, finite-difference spatial operator, and boundary-value helpers. Ticket 03 implemented European Crank-Nicolson validation against closed-form Black-Scholes prices.

Ticket 04 is the first ticket that handles the American option obstacle. American options can be exercised before maturity, so their value must not fall below immediate exercise payoff. This turns the numerical problem from a plain linear system into a constrained problem. Ticket 04 adds the reusable projected SOR logic needed to enforce that constraint.

This ticket is deliberately limited. It implements the PSOR/LCP core and a basic American CN/PSOR time-marching solver. It does not perform the full American put validation study. Ticket 05 will use this core to build validation tables, representative price checks, and deeper obstacle analysis.

2 Theory and Methodology Connection

The source basis for Ticket 04 is internal and project-specific:

- `reports/01_literature/literature_map.md`;
- `reports/02_math/formulation_note.md`;
- `reports/03_solver/solver_validation_plan.md`;
- `reports/03_solver/solver_implementation_tickets.md`;
- the Ticket 01, Ticket 02, and Ticket 03 reports and code.

The literature map identifies variational inequalities, linear complementarity problems, and PSOR as central concepts in classical finite-difference American option pricing. It mentions Brennan-Schwartz finite-difference American put valuation as part of the methodology background, but this report does not create a BibTeX file because the project notes do not yet provide a complete verified bibliography for all external references. The report therefore cites the internal literature map, formulation note, and solver validation plan as the documented source basis.

The methodology connection is stronger in Ticket 04 than in Tickets 01 through 03 because PSOR is where the American exercise right enters the numerical algorithm. Ticket 01 supplied payoff and European closed-form reference functions. Ticket 02 supplied the spatial grid and Black-Scholes operator. Ticket 03 checked the continuation PDE with European Crank-Nicolson. Ticket 04 keeps that same continuation machinery but adds the missing American ingredient: the payoff obstacle.

For a beginner, the key methodological shift is this:

- European validation asks whether the finite-difference PDE machinery can reproduce a closed-form European benchmark.
- American pricing asks a constrained question: at each grid point, is it better to continue or to exercise immediately?
- The finite-difference version of that constrained question is an LCP, and PSOR is the first reusable numerical tool in this project for solving that LCP.

The formulation note describes the American problem as an obstacle problem. The validation plan also emphasizes that later tickets should check obstacle violations and complementarity behavior. Ticket 04

does not yet perform those full validation diagnostics, but it implements the solver core that makes those diagnostics meaningful.

3 Why American Options Become a Variational Inequality and LCP

A European option can be exercised only at maturity. Between today and maturity, its value follows the continuation PDE. In time-to-maturity notation, Ticket 03 solved

$$U_\tau = LU, \tag{1}$$

with terminal condition $U(S, 0) = \Phi(S)$.

An American option is different because the holder can exercise early. At every time and spot point, the holder compares two choices:

- continue holding the option;
- exercise immediately and receive payoff $\Phi(S)$.

The value must therefore satisfy

$$U(S, \tau) \geq \Phi(S). \tag{2}$$

In the continuation region, where holding is better than exercising, the PDE still applies. In the exercise region, the value sits on the payoff. This is why the continuous problem is a variational inequality rather than a plain PDE.

After finite-difference discretization and a Crank-Nicolson time step, the problem becomes a linear complementarity problem, or LCP:

$$Au \geq b, \tag{3}$$

$$u \geq \Phi, \tag{4}$$

$$(u - \Phi)^T (Au - b) = 0. \tag{5}$$

Here u is the vector of new interior-node values, A is the Crank-Nicolson left-hand matrix, b is the adjusted right-hand side, and Φ is the payoff vector on interior nodes.

The three LCP lines each have a plain-language interpretation:

- $Au \geq b$: the discrete continuation equation is not allowed to point to a value that would make immediate exercise preferable.
- $u \geq \Phi$: the option value must stay at or above the exercise payoff.
- $(u - \Phi)^T (Au - b) = 0$: at each node, at least one of the two gaps is active. Either the node is above payoff and behaves like a continuation node, or it sits on payoff and the exercise constraint is binding.

A useful beginner picture is to think of two nonnegative gaps:

$$\text{value gap}_i = u_i - \Phi_i, \qquad \text{equation gap}_i = (Au - b)_i. \tag{6}$$

Complementarity says that their product should be zero at a properly solved node:

$$\text{value gap}_i \times \text{equation gap}_i = 0. \tag{7}$$

This does not mean both gaps must be zero. It means the node should not be simultaneously above payoff and away from the continuation equation.

4 Payoff Obstacle in Plain Language

The payoff obstacle says that an American option cannot be worth less than what the holder can get by exercising immediately. For a put,

$$\Phi_{\text{put}}(S) = \max(K - S, 0). \tag{8}$$

For a call,

$$\Phi_{\text{call}}(S) = \max(S - K, 0). \tag{9}$$

If a numerical solver reports $U < \Phi$, the result is financially invalid. The holder would simply exercise and receive Φ, so a lower computed value cannot be correct. In code, Ticket 04 enforces this by projecting each PSOR update:

$$u_i \leftarrow \max(\Phi_i,\ \text{SOR candidate}_i). \tag{10}$$

This projection is the key difference between the European solver and the American solver. A European solver can accept the linear-system answer because early exercise is not allowed. An American solver must check the linear-system answer against the exercise payoff at every update.

Projection is local and mechanical, but it has a strong financial meaning. It says: after the row calculation proposes a value, the algorithm asks whether an investor could do better by exercising right now. If yes, the node is pulled back up to payoff. If no, the continuation value is allowed to remain above payoff.

Projection alone is not a full validation proof. It protects against direct obstacle violation, but later tickets still need to examine accuracy, complementarity residuals, and financial benchmarks.

5 Continuation Value Versus Immediate Exercise Value

The continuation value is the value of keeping the option alive. The immediate exercise value is the payoff. American pricing asks which is better at every grid point.

If continuation is better, then

$$u_i > \Phi_i \tag{11}$$

and the finite-difference equation should hold closely:

$$(Au - b)_i \approx 0. \tag{12}$$

If immediate exercise is better, then

$$u_i = \Phi_i \tag{13}$$

and the PDE equation is not forced in the same way. The complementarity condition in Equation 5 expresses this either-or logic. Each point should either behave like a continuation point or sit on the obstacle.

Region	Mathematical signal	Plain-language meaning
Continuation region	$u_i > \Phi_i$ and $(Au - b)_i \approx 0$	Holding the option is better than exercising. The node behaves like the European continuation equation.

Region	Mathematical signal	Plain-language meaning
Exercise region	$u_i = \Phi_i$ and $(Au - b)_i \geq 0$	Immediate exercise is binding. The value is pinned to payoff, so the continuation equation is not the active condition.
Near the boundary	$u_i - \Phi_i$ is small and numerical tolerances matter	The grid point is close to the exercise frontier. Later tickets should interpret diagnostics with tolerance rather than exact equality.

For an American put, early exercise can become attractive when the spot price is low enough. For an American call with dividends, early exercise can become structurally relevant because receiving stock earlier may matter when dividends are present. Ticket 04 supports both paths structurally, but it does not yet claim full validation for either case.

6 PSOR and Projection in Plain Language

Successive over-relaxation, or SOR, is an iterative way to solve a linear system. It updates one row at a time and uses the newest values as soon as they are available. The relaxation parameter ω controls how strongly the solver moves toward the row solution:

$$u_i^{\mathrm{SOR}} = u_i^{\mathrm{old}} + \omega \left(u_i^{\mathrm{GS}} - u_i^{\mathrm{old}} \right), \tag{14}$$

where u_i^{GS} is the ordinary Gauss-Seidel row candidate.

Projected SOR adds one extra step:

$$u_i^{\mathrm{new}} = \max \left(\Phi_i,\ u_i^{\mathrm{SOR}} \right). \tag{15}$$

In beginner language, each row asks: “What value would the linear equation suggest?” Then the projection asks: “Is that value below payoff?” If it is below payoff, the update is pushed back to payoff.

Component	Role	Beginner interpretation
A	CN left-hand matrix	The continuation equation to solve at the new time level.
b	Adjusted right-hand side	Old value information plus boundary adjustments.
Φ	Payoff vector	The minimum allowed American option value.
ω	Relaxation parameter	How aggressively the row update moves.
Projection	$u_i = \max(\Phi_i, \cdot)$	The safeguard that enforces the obstacle.
Convergence metadata	Iterations and final update	Evidence about whether PSOR actually settled.

6.1 Step-by-Step PSOR Algorithm Walkthrough

The projected SOR solve used in Ticket 04 is easier to understand as a repeated sweep through the interior grid:

1. Start from an initial guess. In the time-marching solver, the previous time-level values are a natural starting point.
2. Sweep through the LCP rows from left to right.
3. For the current row i, compute the Gauss-Seidel row candidate. The left-neighbour value u_{i-1} has already been updated in the current sweep, so the newest available left value is used. The right-neighbour value u_{i+1} has not yet been visited in the current sweep, so its current stored value is used.
4. Apply SOR relaxation. If $\omega = 1$, this is ordinary Gauss-Seidel. If $1 < \omega < 2$, the update moves more aggressively toward the row candidate.
5. Project above payoff:
$$u_i \leftarrow \max(\Phi_i, u_i^{\mathrm{SOR}}).$$
This is the American-option step.
6. Track the largest absolute change made during the sweep.
7. Stop when the largest update is less than the requested tolerance.
8. If the maximum number of sweeps is reached first, report non-convergence instead of silently treating the answer as reliable.

PSOR stage	Numerical action	Beginner debugging question
Initial guess	Copy the supplied initial vector or start from payoff.	Is the starting vector the right length and above payoff when expected?
Row candidate	Solve one tridiagonal row using latest left and current right neighbour values.	Are the neighbour indices aligned with the matrix rows?
Relaxation	Blend old value and Gauss-Seidel candidate using ω.	Is $0 < \omega < 2$?
Projection	Replace the candidate by $\max(\Phi_i, \cdot)$.	Can any node fall below payoff after the update?
Stopping check	Measure the maximum absolute row update.	Is non-convergence reported if the tolerance is not reached?

7 How Ticket 04 Extends Ticket 03 European CN

Ticket 03 used Crank-Nicolson in the European continuation case:

$$\left(I - \frac{\Delta\tau}{2}L_h\right)U^{n+1} = \left(I + \frac{\Delta\tau}{2}L_h\right)U^n. \tag{16}$$

Ticket 04 keeps the same matrix and boundary logic. It still builds

$$A = I - \frac{\Delta\tau}{2}L_h \tag{17}$$

and

$$b^n = \left(I + \frac{\Delta\tau}{2} L_h\right) U^n \tag{18}$$

with new-time boundary adjustments included in b^n.

The change is the solve. Ticket 03 solved the unconstrained European linear system. Ticket 04 solves the constrained American LCP with PSOR:

$$AU^{n+1} \geq b^n, \tag{19}$$

$$U^{n+1} \geq \Phi. \tag{20}$$

The Ticket 04 solver stores the full value grid over (τ, S) so later tickets can compute obstacle diagnostics, continuation premiums, validation tables, and boundary extraction. Ticket 04 itself does not compute those later diagnostics.

One CN/PSOR time-step stage	What the code is doing	Why it matters
Old value grid	Start with U^n, the option values already computed at the previous time-to-maturity level.	This is the known information for the next step.
Old boundary values	Use boundary formulas at τ_n when building the right-hand side.	Boundary nodes affect nearby interior nodes through the spatial operator.
Right-hand side construction	Form $(I + \frac{\Delta\tau}{2} L_h)U^n$ on the interior nodes.	This carries old-time continuation information forward.
New boundary adjustment	Move known boundary contributions from AU^{n+1} into the right-hand side.	The LCP solve handles only interior unknowns, so boundary terms must be accounted for explicitly.
LCP solve by PSOR	Solve $Au \geq b$, $u \geq \Phi$ for the new interior vector.	This is where American exercise enters the time step.
Projection against payoff	Each row update is clipped upward to payoff.	This enforces the obstacle locally.
Metadata storage	Store convergence status, iteration count, tolerance, ω, and final update.	Later validation can detect slow or failed PSOR steps.

8 LCP Matrix Form and Update Logic

For a tridiagonal row,

$$\ell_i u_{i-1} + d_i u_i + r_i u_{i+1} = b_i, \tag{21}$$

where ℓ_i, d_i, and r_i are the lower, diagonal, and upper matrix entries.

The Gauss-Seidel row candidate is

$$u_i^{\mathrm{GS}} = \frac{b_i - \ell_i u_{i-1}^{\mathrm{new}} - r_i u_{i+1}^{\mathrm{old}}}{d_i}. \tag{22}$$

Ticket 04 then applies relaxation and projection:

$$u_i^{\mathrm{SOR}} = u_i^{\mathrm{old}} + \omega(u_i^{\mathrm{GS}} - u_i^{\mathrm{old}}), \tag{23}$$

$$u_i^{\mathrm{new}} = \max(\Phi_i, u_i^{\mathrm{SOR}}). \tag{24}$$

The implementation uses the maximum absolute update as a convergence proxy:

$$\max_i |u_i^{\mathrm{new}} - u_i^{\mathrm{old}}|. \tag{25}$$

When this quantity is below the requested tolerance, the PSOR solve reports convergence. If the maximum iteration count is reached first, non-convergence is reported explicitly.

The words "old" and "new" are local to a PSOR sweep. During a single sweep, nodes already visited on the left use their newest values, while nodes not yet visited on the right still use their current stored values. This mixed use of newest and current values is not a bug; it is the Gauss-Seidel idea that makes SOR different from a fully simultaneous Jacobi update.

9 Implementation Design Decisions

9.1 One Focused Module

Ticket 04 creates one source module, `src/american_risk_surfaces/solvers/cn_psor.py`. It contains only the PSOR/LCP core and American CN/PSOR time-marching core. It does not create diagnostic, boundary-extraction, Greek, experiment, dataset, stress-map, or neural-network modules.

9.2 Interior Nodes Only

The PSOR solver works on interior nodes only. Boundary values are imposed outside the LCP solve, consistent with Tickets 02 and 03. This keeps the distinction clear: interior nodes are solved, boundary nodes are assigned by boundary formulas.

9.3 Metadata Is Part of the Result

PSOR can fail to converge if the tolerance, relaxation parameter, grid, or maximum iteration count is poor. Ticket 04 therefore returns convergence metadata rather than hiding this risk.

Metadata field	Meaning
`converged`	Whether the final update fell below tolerance before `max_iter`.
`iterations`	Number of PSOR sweeps used.
`final_update`	Maximum absolute update in the final sweep.
`tolerance`	Requested stopping tolerance.
`omega`	Relaxation parameter used in the solve.
`max_iter`	Maximum number of sweeps allowed.

9.4 American Put and Dividend-Call Structure

The time-marching solver supports American puts and American calls. It uses Ticket 01 payoff functions and Ticket 02 American boundary helpers. The call path is structural at this stage: it runs with dividend-

aware call boundaries, but detailed no-dividend and dividend-paying call validation belongs to later tickets.

10 Code-Level Function Explanations

Function or class	Purpose and important behavior
`PSORResult`	Frozen dataclass containing one LCP solution and convergence metadata.
`AmericanCNPSORResult`	Frozen dataclass containing option parameters, grids, payoff, full value grid, final values, per-step PSOR metadata, convergence status, and maximum obstacle violation.
`psor_lcp_solve`	Solves a tridiagonal LCP on interior nodes using projected SOR. It validates array shapes, ω, tolerance, and max iterations.
`american_crank_nicolson_psor_price`	Builds the grid and CN matrices, forms each right-hand side, solves each American time step with PSOR, and stores the value grid.

The public API is controlled through `__all__`. Boundary helpers from Ticket 02 are used internally but not exported as Ticket 04 public API.

11 Testing Strategy

Test category	What it proves	What it does not prove
Toy LCP projection	Projected SOR pushes values back above payoff in a known small system.	Full option-pricing accuracy.
American put smoke test	The basic put time-marching path preserves $U \geq \Phi$ in a small case.	Ticket 05 American put validation.
Metadata test	Convergence fields are exposed and populated.	That every parameter choice converges quickly.
Non-convergence test	Failure to converge is reported rather than hidden.	How to tune ω for production grids.
$T = 0$ test	The solver returns payoff at maturity without PSOR steps.	Positive-maturity American behavior.
Dividend call structure test	The call path runs with call payoff and call-style lower boundary.	No-dividend or dividend-paying call validation.
Invalid input tests	Bad option type, grid inputs, ω, tolerance, and LCP shapes raise `ValueError`.	Every possible production input policy.

Test category	What it proves	What it does not prove
Scope test	The public API does not add boundary extraction, Greeks, stress, dataset, or neural surfaces.	Future tickets that intentionally add those features.

12 Test Results and Interpretation

The exact test command was:

```
PYTHONPYCACHEPREFIX=/private/tmp/codex_pycache PYTHONPATH=src python3 -m unittest
    discover -s tests
```

Observed result after implementing Ticket 04:

```
Ran 40 tests in 0.030s

OK
```

The compile command also completed successfully:

```
PYTHONPYCACHEPREFIX=/private/tmp/codex_pycache PYTHONPATH=src python3 -m compileall
    -q src tests experiments
```

For the American put smoke case used by the tests, the solver reported convergence across 40 time steps, maximum obstacle violation 0, and no hidden non-convergence. This is a useful local correctness check for the PSOR core. It is not a full validation study.

Observed outcome	Recorded result	Interpretation
Total unittest count	40 tests passed	The Ticket 04 tests ran together with the earlier Ticket 01–03 tests.
American put smoke case	Converged across 40 time steps	The basic American put CN/PSOR path completed every step in the small test case.
Maximum obstacle violation	0	No tested grid value fell below payoff in the American put smoke case.
Non-convergence case	Reported explicitly	A deliberately constrained PSOR run returned `converged=False` rather than hiding the failure.
$T = 0$ behavior	Returned payoff	At maturity, the American option value equals immediate exercise payoff and no time marching is needed.

These outcomes are important, but narrow. They show that the implementation respects the main mechanical requirements of Ticket 04. They do not establish production accuracy for American puts, do not compare against external reference values, and do not prove PSOR convergence for all parameter choices.

13 Implementation Pitfalls and Debugging Notes

Ticket 04 recorded one expected test-driven red step and one workflow-level import issue. Neither was a mathematical failure of the PSOR algorithm, but both are useful for beginner researchers to understand because they show how implementation checks can fail for ordinary engineering reasons.

Recorded item or pitfall	Symptom	Fix or guard	Beginner lesson
Expected TDD red run before implementation	`ModuleNotFoundError` for the missing `cn_psor` module.	Create the Ticket 04 module and public functions, then rerun the same tests.	A red test is useful when it proves the test targets the missing API.
Ad-hoc metadata probe without `PYTHONPATH=src`	Package import failed even though the source tree existed.	Rerun using the project test convention `PYTHONPATH=src`.	Import failures can be environment issues, not code logic issues.
Mixing PSOR neighbour timing incorrectly	Updates may converge slowly or solve the wrong row equation.	Use newest left-neighbour values and current right-neighbour values during each sweep.	Gauss-Seidel/SOR is sequential, not simultaneous.
Wrong boundary adjustment sign	Interior values near boundaries can be distorted.	Reuse the Ticket 03 CN boundary-adjustment pattern carefully.	Boundary terms are known values, but they still affect the interior system.
Forgetting projection above payoff	The computed American value can fall below Φ.	Apply $u_i = \max(\Phi_i, \text{candidate})$ at every row update.	Projection is the American-option part of PSOR.
Hiding non-convergence	The solver may look successful after reaching `max_iter`.	Return `converged=False`, iteration count, and final update.	Numerical algorithms need honest status metadata.
Using put payoff in the call path	The dividend-call structure would run with the wrong exercise value.	Test the call path separately with call payoff and call-style boundaries.	American puts and calls share machinery but not payoff logic.
Treating Ticket 04 smoke tests as full validation	Small tests could be overinterpreted as solver accuracy evidence.	State clearly that full American put validation belongs to Ticket 05.	Passing smoke tests is necessary, but not enough for model credibility.
Overclaiming from a small smoke test	The report could imply the whole American solver is validated.	Keep limitations explicit and avoid claiming benchmark accuracy.	Good computational finance reports separate implementation checks from validation studies.

Future coding and testing reports should include an “Implementation Pitfalls and Debugging Notes”

section when useful, especially if failed tests, environment mistakes, numerical bugs, or fixes occurred during implementation. This makes the reports more valuable as study material instead of only recording the final clean state.

14 Limitations

Not included in Ticket 04	Reason
Full American put validation	Belongs to Ticket 05.
No-dividend American call validation	Belongs to a later call-validation ticket.
Dividend-paying American call validation	Belongs to a later dividend-call ticket.
Boundary extraction	Requires continuation-premium logic planned for Ticket 09.
Greeks	Delta and Gamma diagnostics are later work.
Stress maps, datasets, neural networks	These depend on a fully validated solver.
Figures	Not essential for this core-code ticket.
Formal PSOR convergence proof	Background theory, not part of this implementation ticket.

The current implementation uses a final-update stopping proxy. Later diagnostic tickets may add more detailed complementarity residual calculations and reporting.

15 Lessons Learned / Study Notes

The first lesson is that American option pricing is constrained pricing. The value is not allowed to fall below payoff because the holder can exercise immediately.

The second lesson is that the LCP is the discrete version of the exercise decision. A node is either a continuation node, where the equation is active, or an exercise node, where the payoff obstacle is active.

The third lesson is that PSOR is not just a linear solver with a different name. Projection is the important American-option step. Without projection, the method would only be a European continuation solve.

The fourth lesson is that convergence metadata matters. If PSOR stops too early or reaches the maximum iteration count, the solver output should say so directly.

The fifth lesson is to keep validation layered. Ticket 04 checks the reusable PSOR mechanism and basic American time-marching structure. It does not replace the detailed American put validation that comes next.

16 Link to Ticket 05 American Put Validation

Ticket 05 should use the Ticket 04 solver to run a detailed American put validation study. That next ticket should add validation tables, selected price checks, obstacle summaries, and more financial interpretation for the put case. It should still avoid boundary extraction and Greeks unless the approved roadmap explicitly moves to those later tickets.

The handoff from Ticket 04 to Ticket 05 is:

- Ticket 04 provides the American CN/PSOR solver and metadata.
- Ticket 05 should evaluate whether the American put results are numerically and financially credible.

17 Reproducibility Record

17.1 Files Created

Path	Role in Ticket 04
`src/american_risk_surfaces/solvers/cn_psor.py`	PSOR/LCP core and basic American CN/PSOR time-marching solver.
`tests/test_psor.py`	Focused `unittest` coverage for projected SOR, American smoke behavior, metadata, non-convergence, invalid inputs, and scope.
`reports/03_solver/tickets/ticket_04_psor_lcp_core.tex`	Formal LaTeX study-report source.
`reports/03_solver/tickets/ticket_04_psor_lcp_core.pdf`	Formal PDF compiled from the LaTeX source.

17.2 Commands

Test command:

```
PYTHONPYCACHEPREFIX=/private/tmp/codex_pycache PYTHONPATH=src python3 -m unittest
    discover -s tests
```

Python compile command:

```
PYTHONPYCACHEPREFIX=/private/tmp/codex_pycache PYTHONPATH=src python3 -m compileall
    -q src tests experiments
```

LaTeX compile command:

```
HOME=/private/tmp FC_CACHEDIR=/private/tmp/fontconfig-cache latexmk -xelatex -
    interaction=nonstopmode -halt-on-error -outdir=/private/tmp/ticket04_latex
    reports/03_solver/tickets/ticket_04_psor_lcp_core.tex
```

PDF verification uses `pdfinfo` and page rendering with `pdftoppm`. Raw Git status is intentionally not included in this formal PDF report.